# Lab Eleven – Point Estimation and Resampling

**Tips**

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

# 1 Introduction

When performing point estimation, there isn't just one *right* answer. For example, the Method of Moments (MOM) and Maximum Likelihood Estimates (MLEs) aren't always the same.

One question that comes up, is why we calculate the sample variance as

$$S^2_{n-1} = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2,$$

when it is supposed to be the "average squared distance from the mean, i.e.,

$$S^2_n = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

# 2 Comparing Estimators

## 2.1 Initial Simulation

In this lab, we will explore why we use $S^2_{n-1}$ and not $S^2_n$. To do this, we will generate data from four known distributions:

- Poisson$(\lambda = 2)$
- Binomial$\left(n = 50, p = \frac{5-\sqrt{21}}{10}\right)$
- Normal$(\mu = 0, \sigma = \sqrt{2})$
- Exponential$\left(\lambda = \frac{1}{\sqrt{2}}\right)$.

**Remark**: Note that I have selected the parameters so that the true variance for all four models is 2.

For each distribution, write a loop that completes the following 1000 times. Put `set.seed(7272)` at the top of your code chunk.

- Generate a sample of size $n$ from the distribution.
- Calculate and save $S^2_{n-1}$ and $S^2_n$.

Start with $n = 20$. Is there evidence that $S^2_{n-1}$ is a better estimator than $S^2_n$?

Create a plot or combination of plots (using patchwork) to show the results across all four distributions. Make sure to create a corresponding table that numerically summarizes the results of the simulation. You should include

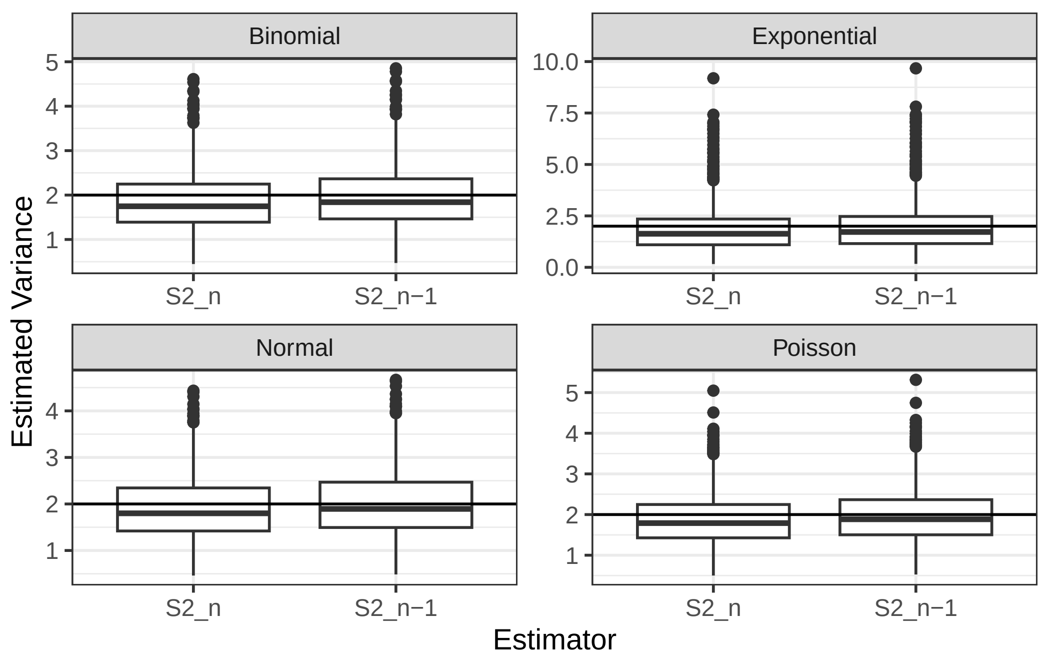
$$\text{Bias} = \text{Average of Estimates} - \text{True Value}$$
$$\text{Precision} = \frac{1}{\text{Variance of Estimates}}$$
$$\text{Mean Square Error} = \text{Bias}^2 + \text{Variance of Estimates.}$$

**Solution**

```r
set.seed(7272)

#numbers
n = 20
true_val = 2

pois_n = rep(NA, 1000)
pois_n1 = rep(NA, 1000)

for (i in 1:1000){
  x_pois = rpois(n, 2)
  pois_n1[i] = var(x_pois)
  pois_n[i] = (n-1)/n * var(x_pois)
}

binom_n = rep(NA, 1000)
binom_n1 = rep(NA, 1000)

for (i in 1:1000){
  x_binom = rbinom(n, size = 50, prob = (5-sqrt(21))/10)
  binom_n1[i] = var(x_binom)
  binom_n[i] = (n-1)/n * var(x_binom)
}

norm_n = rep(NA, 1000)
norm_n1 = rep(NA, 1000)

for (i in 1:1000){
  x_norm = rnorm(n, mean = 0, sd = sqrt(2))
  norm_n1[i] = var(x_norm)
  norm_n[i] = (n-1)/n * var(x_norm)
}

expo_n = rep(NA, 1000)
expo_n1 = rep(NA, 1000)

for (i in 1:1000){
  x_expo = rexp(n, rate = sqrt(1/2)) #dis is the same as 1/sqrt(2) but im cool like that
  expo_n1[i] = var(x_expo)
  expo_n[i] = (n-1)/n * var(x_expo)
}

results = bind_rows(
  tibble(distribution ="Poisson", estimator ="S2_n-1", estimate =pois_n1),
  tibble(distribution ="Poisson", estimator ="S2_n", estimate =pois_n),
  tibble(distribution ="Binomial", estimator ="S2_n-1", estimate =binom_n1),
  tibble(distribution ="Binomial", estimator ="S2_n", estimate =binom_n),
```

```r
  tibble(distribution ="Normal", estimator ="S2_n-1", estimate =norm_n1),
  tibble(distribution ="Normal", estimator ="S2_n", estimate =norm_n),
  tibble(distribution ="Exponential", estimator ="S2_n-1", estimate =expo_n1),
  tibble(distribution ="Exponential", estimator ="S2_n", estimate =expo_n)
)

ggplot(results, aes(x =estimator, y =estimate)) +
  geom_boxplot() +
  facet_wrap(~distribution, scales = "free")+
  geom_hline(yintercept=true_val)+
  xlab("Estimator") +
  ylab("Estimated Variance")+
  theme_bw()
```

Figure 1: Variance Estimators by Distribution




```r
summary_table = results |>
  group_by(distribution, estimator) |>
  summarize(
    bias = mean(estimate) - true_val,
    precision = 1/(var(estimate)),
    mse = bias^2 + var(estimate),
    average_estimate = mean(estimate), #added these bc relevant to simulations
    variance_of_estimates = var(estimate),
  )
```

```r
knitr::kable(summary_table, digits = 4)
```

| distribution | estimator | bias | precision | mse | average_estimate | variance_of_estimates |
|---|---|---|---|---|---|---|
| Binomial | S2_n | -0.1336 | 2.3821 | 0.4376 | 1.8664 | 0.4198 |
| Binomial | S2_n-1 | -0.0354 | 2.1498 | 0.4664 | 1.9646 | 0.4652 |
| Exponential | S2_n | -0.1150 | 0.7746 | 1.3042 | 1.8850 | 1.2910 |
| Exponential | S2_n-1 | -0.0158 | 0.6991 | 1.4307 | 1.9842 | 1.4305 |

| distribution | estimator | bias | precision | mse | average_estimate | variance_of_estimates |
|---|---|---|---|---|---|---|
| Normal | S2_n | -0.0978 | 2.2480 | 0.4544 | 1.9022 | 0.4448 |
| Normal | S2_n-1 | 0.0023 | 2.0289 | 0.4929 | 2.0023 | 0.4929 |
| Poisson | S2_n | -0.1153 | 2.3283 | 0.4428 | 1.8847 | 0.4295 |
| Poisson | S2_n-1 | -0.0161 | 2.1013 | 0.4762 | 1.9839 | 0.4759 |

Based on our above figure, we can see that for most distributions $S^2_{n-1}$ is generally closer to the true variance value compared to the other estimator, $S^2_n$. We can generally keep an eye on this as evidence for our best estimator as of yet.

## 2.2 Simulation over various sample sizes

Consider the effect of the sample size have on these results. Provide a graphical and numerical summary of what happens as $n$ increases – say $n = 20$, $n = 50$, $n = 100$, and $n = 500$. Does the result in your initial simulation hold across samples? Or, do things change as the sample size changes? Ensure to set your randomization seed.

**Solution**

```r
set.seed(7272)

simulate_by_size = function(n){

  pois_n = rep(NA, 1000)
  pois_n1 = rep(NA, 1000)

  for (i in 1:1000){
    x_pois = rpois(n, 2)
    pois_n1[i] = var(x_pois)
    pois_n[i] = (n-1)/n * var(x_pois)
  }

  binom_n = rep(NA, 1000)
  binom_n1 = rep(NA, 1000)

  for (i in 1:1000){
    x_binom = rbinom(n, size = 50, prob = (5-sqrt(21))/10)
    binom_n1[i] = var(x_binom)
    binom_n[i] = (n-1)/n * var(x_binom)
  }

  norm_n = rep(NA, 1000)
  norm_n1 = rep(NA, 1000)

  for (i in 1:1000){
    x_norm = rnorm(n, mean = 0, sd = sqrt(2))
    norm_n1[i] = var(x_norm)
    norm_n[i] = (n-1)/n * var(x_norm)
  }

  expo_n = rep(NA, 1000)
  expo_n1 = rep(NA, 1000)

  for (i in 1:1000){
    x_expo = rexp(n, rate = sqrt(1/2)) #dis is the same as 1/sqrt(2) but im cool like that
    expo_n1[i] = var(x_expo)
    expo_n[i] = (n-1)/n * var(x_expo)
  }
```

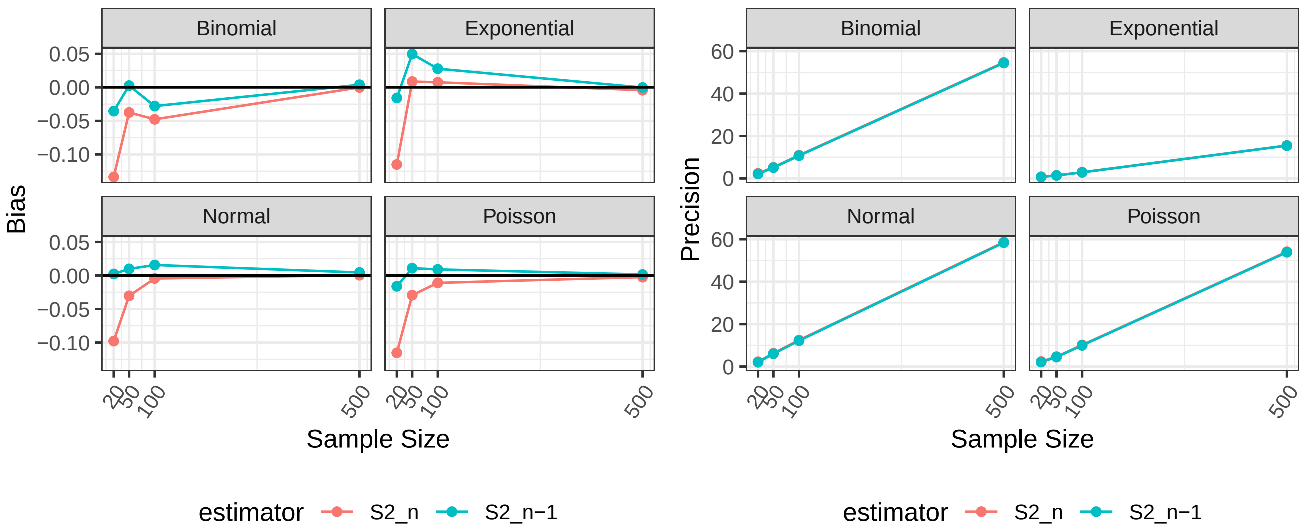
```r

  results = bind_rows(
    tibble(sample_size = n,distribution ="Poisson", estimator ="S2_n-1", estimate =pois_n1),
    tibble(sample_size = n,distribution ="Poisson", estimator ="S2_n", estimate =pois_n),
    tibble(sample_size = n,distribution ="Binomial", estimator ="S2_n-1", estimate =binom_n1),
    tibble(sample_size = n,distribution ="Binomial", estimator ="S2_n", estimate =binom_n),
    tibble(sample_size = n,distribution ="Normal", estimator ="S2_n-1", estimate =norm_n1),
    tibble(sample_size = n,distribution ="Normal", estimator ="S2_n", estimate =norm_n),
    tibble(sample_size = n,distribution ="Exponential", estimator ="S2_n-1", estimate =expo_n1),
    tibble(sample_size = n,distribution ="Exponential", estimator ="S2_n", estimate =expo_n)
  )
  return(results)
}

all_results = bind_rows(
  simulate_by_size(20),
  simulate_by_size(50),
  simulate_by_size(100),
  simulate_by_size(500)
)

summary_table = all_results |>
  group_by(sample_size, distribution, estimator) |>
  summarize(
    bias = mean(estimate) - true_val,
    precision = 1 / var(estimate),
    mse = bias^2 + var(estimate),
    average_estimate = mean(estimate),
    variance_of_estimates = var(estimate)
  )

knitr::kable(summary_table, digits = 4)
```

| sample_size | distribution | estimator | bias | precision | mse | average_estimate | variance_of_estimates |
|---:|---|---|---:|---:|---:|---:|---:|
| 20 | Binomial | S2_n | -0.1336 | 2.3821 | 0.4376 | 1.8664 | 0.4198 |
| 20 | Binomial | S2_n-1 | -0.0354 | 2.1498 | 0.4664 | 1.9646 | 0.4652 |
| 20 | Exponential | S2_n | -0.1150 | 0.7746 | 1.3042 | 1.8850 | 1.2910 |
| 20 | Exponential | S2_n-1 | -0.0158 | 0.6991 | 1.4307 | 1.9842 | 1.4305 |
| 20 | Normal | S2_n | -0.0978 | 2.2480 | 0.4544 | 1.9022 | 0.4448 |
| 20 | Normal | S2_n-1 | 0.0023 | 2.0289 | 0.4929 | 2.0023 | 0.4929 |
| 20 | Poisson | S2_n | -0.1153 | 2.3283 | 0.4428 | 1.8847 | 0.4295 |
| 20 | Poisson | S2_n-1 | -0.0161 | 2.1013 | 0.4762 | 1.9839 | 0.4759 |
| 50 | Binomial | S2_n | -0.0374 | 5.2558 | 0.1917 | 1.9626 | 0.1903 |
| 50 | Binomial | S2_n-1 | 0.0026 | 5.0477 | 0.1981 | 2.0026 | 0.1981 |
| 50 | Exponential | S2_n | 0.0087 | 1.4557 | 0.6870 | 2.0087 | 0.6869 |
| 50 | Exponential | S2_n-1 | 0.0497 | 1.3981 | 0.7177 | 2.0497 | 0.7153 |
| 50 | Normal | S2_n | -0.0304 | 6.2500 | 0.1609 | 1.9696 | 0.1600 |
| 50 | Normal | S2_n-1 | 0.0098 | 6.0025 | 0.1667 | 2.0098 | 0.1666 |
| 50 | Poisson | S2_n | -0.0293 | 4.6500 | 0.2159 | 1.9707 | 0.2151 |
| 50 | Poisson | S2_n-1 | 0.0109 | 4.4659 | 0.2240 | 2.0109 | 0.2239 |
| 100 | Binomial | S2_n | -0.0476 | 10.9444 | 0.0936 | 1.9524 | 0.0914 |
| 100 | Binomial | S2_n-1 | -0.0279 | 10.7266 | 0.0940 | 1.9721 | 0.0932 |
| 100 | Exponential | S2_n | 0.0076 | 2.9136 | 0.3433 | 2.0076 | 0.3432 |
| 100 | Exponential | S2_n-1 | 0.0278 | 2.8556 | 0.3510 | 2.0278 | 0.3502 |
| 100 | Normal | S2_n | -0.0045 | 12.4465 | 0.0804 | 1.9955 | 0.0803 |

| sample_size | distribution | estimator | bias | precision | mse | average_estimate | variance_of_estimates |
|---|---|---|---|---|---|---|---|
| 100 | Normal | S2_n-1 | 0.0157 | 12.1988 | 0.0822 | 2.0157 | 0.0820 |
| 100 | Poisson | S2_n | -0.0110 | 10.1671 | 0.0985 | 1.9890 | 0.0984 |
| 100 | Poisson | S2_n-1 | 0.0091 | 9.9648 | 0.1004 | 2.0091 | 0.1004 |
| 500 | Binomial | S2_n | -0.0001 | 54.6630 | 0.0183 | 1.9999 | 0.0183 |
| 500 | Binomial | S2_n-1 | 0.0039 | 54.4446 | 0.0184 | 2.0039 | 0.0184 |
| 500 | Exponential | S2_n | -0.0042 | 15.5185 | 0.0645 | 1.9958 | 0.0644 |
| 500 | Exponential | S2_n-1 | -0.0002 | 15.4565 | 0.0647 | 1.9998 | 0.0647 |
| 500 | Normal | S2_n | 0.0005 | 58.6278 | 0.0171 | 2.0005 | 0.0171 |
| 500 | Normal | S2_n-1 | 0.0045 | 58.3935 | 0.0171 | 2.0045 | 0.0171 |
| 500 | Poisson | S2_n | -0.0025 | 54.1056 | 0.0185 | 1.9975 | 0.0185 |
| 500 | Poisson | S2_n-1 | 0.0015 | 53.8894 | 0.0186 | 2.0015 | 0.0186 |

```r
bias_plot = ggplot(summary_table,
                 aes(x = sample_size, y = bias, color = estimator)) +
  geom_line() +
  geom_point() +
  facet_wrap(~distribution) +
  xlab("Sample Size") +
  ylab("Bias")+
  geom_hline(yintercept=0)+
  scale_x_continuous(breaks = c(20, 50, 100, 500)) +
  theme_bw()+
  theme( #had to look this up so it wasnt in the way and the x intercept was visible
    legend.position = "bottom",
    axis.text.x = element_text(angle = 55, hjust = 1)
  )

precision_plot = ggplot(summary_table,
                      aes(x = sample_size, y = precision, color = estimator)) +
  geom_line() +
  geom_point() +
  facet_wrap(~distribution) +
  xlab("Sample Size")+
  ylab("Precision")+
  scale_x_continuous(breaks = c(20, 50, 100, 500)) +
  theme_bw() +
  theme(
    legend.position = "bottom",
    axis.text.x = element_text(angle = 55, hjust = 1)
  )

bias_plot | precision_plot
```

Figure 2: Sample Size vs Bias and Precision




Looking first at our precision graphs, we see that both estimators evolve almost exactly the same when scaling with sample size, even having extremely similar values themselves for our precicion measurement. Thus, we disregard the precision graph and use the bias graph to make a proper distinction between each estimator. We see that generally $S^2_{n-1}$ has lower bias compared to $S^2_n$ at every low sample size, thus we can say that it is a better estimator at low sample sizes. At large sample sizes, both estimators seem to converge very close to 0 bias, thus using either estimator at large sample sizes may not actually matter much.

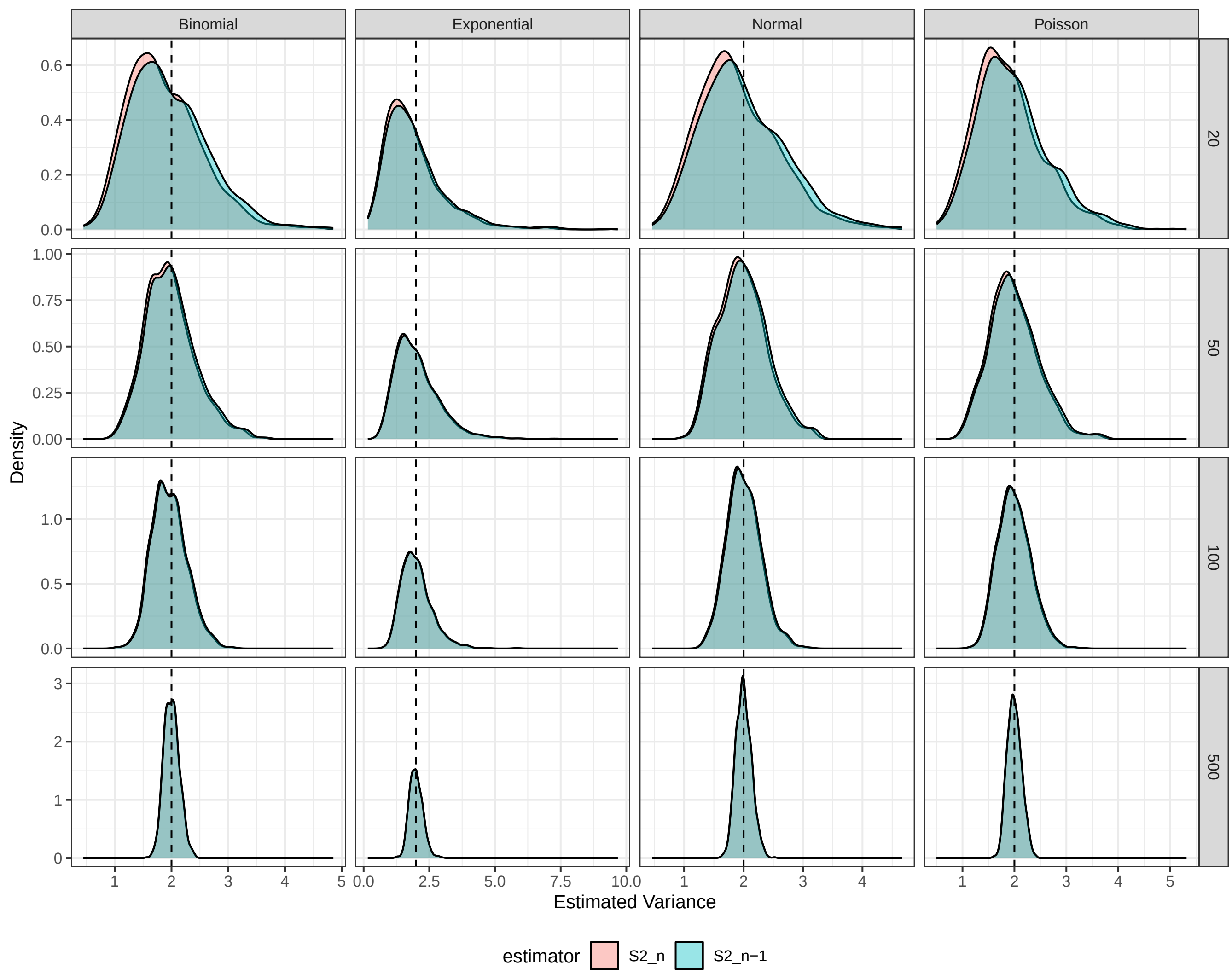
### 2.3 Simulation

You have written code to draw a random sample of various sizes from each distribution, and to calculate and store the calculated estimates. Rework your code to produce a density plot of the $s^2_{n-1}$ and $s^2_n$. Do you have a guess about their distribution? Ensure to set your randomization seed.

**Solution**

```
set.seed(7272)

ggplot(all_results, aes(x = estimate, fill = estimator)) +
  geom_density(alpha = 0.4) +
  facet_grid(sample_size~distribution, scales = "free") + #goated oscar refine
  geom_vline(xintercept = true_val, linetype = "dashed") +
  xlab("Estimated Variance") +
  ylab("Density") +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
```

Figure 3: Density Plots of Variance Estimators with Varying Sample Sizes




Here, we see that at lower values of n, $S^2_{n-1}$ is more centered around the true variance value. We also see the at higher n values, both estimators share the same shape and are pretty gaussian distributed around the true value. I could have guessed that at lower n values, the density plot was going to share similarities with a chi squared plot as we are sampling on variance, which implies a chi squared distribution with right skewness. This is exactly what we see here.

## 2.4 Bootstrap

Now, take *one* sample from each distribution. Instead of drawing a random sample at each iteration, sample *with replacement* from the one sample. Produce a density plot of the $s^2_{n-1}$ and $s^2_n$. Do we get roughly the same information as we do in the full simulation? Try this over several seeds – what do you notice? Ensure to set your randomization seed.

**Solution**

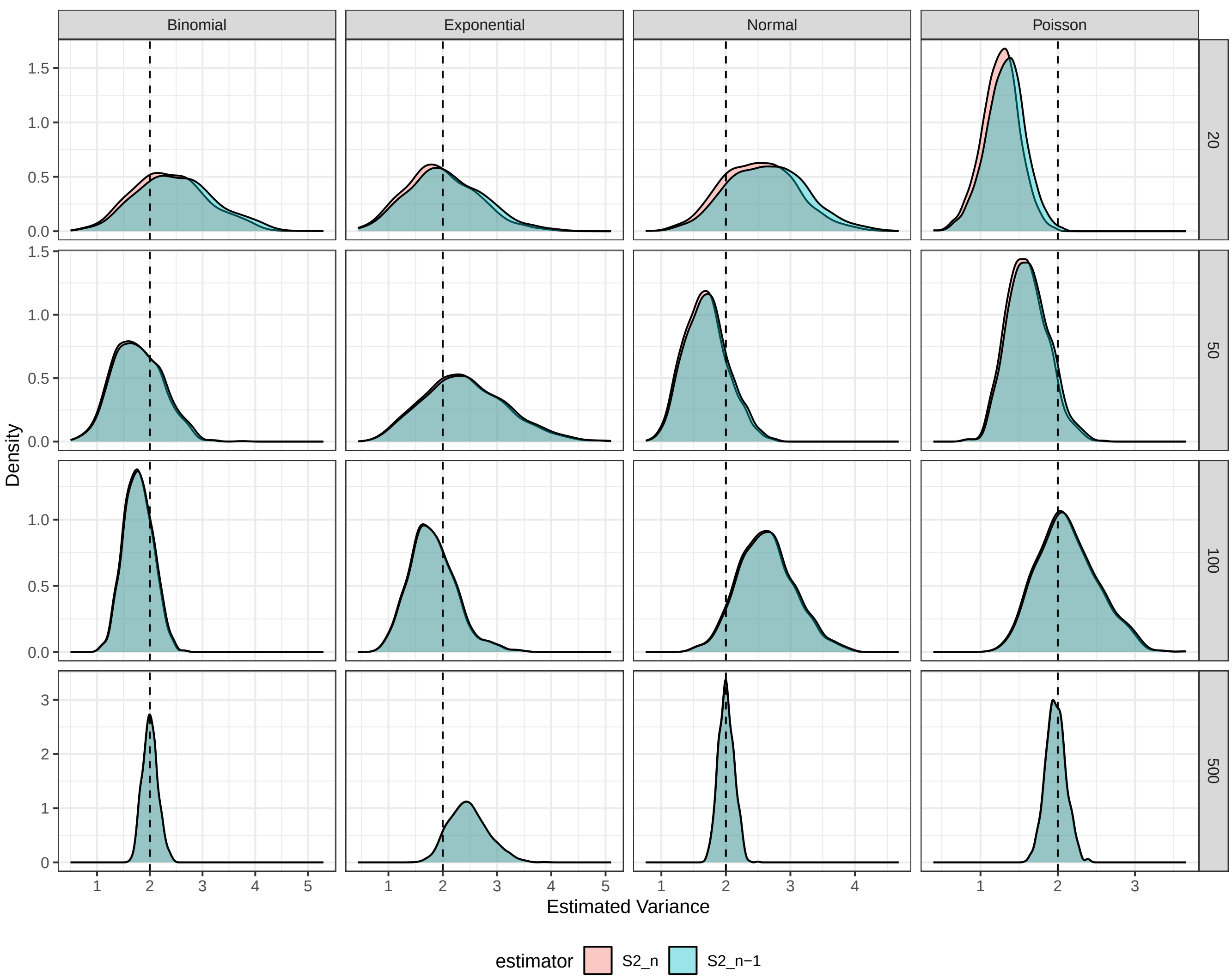
```
set.seed(7272)

boot_by_size = function(n){

#distributions
  x_pois = rpois(n, 2)
```

```
  x_binom = rbinom(n, size = 50, prob = (5-sqrt(21))/10)
  x_norm = rnorm(n, mean = 0, sd = sqrt(2))
  x_expo = rexp(n, rate = sqrt(1/2))

#storage
  pois_n = rep(NA, 1000)
  pois_n1 = rep(NA, 1000)

#booting here
  for (i in 1:1000){
    boot_pois = sample(x_pois, size = n, replace = TRUE)
    pois_n1[i] = var(boot_pois)
    pois_n[i] = (n-1)/n * var(boot_pois)
  }

#rinse repeat for each distr
  binom_n = rep(NA, 1000)
  binom_n1 = rep(NA, 1000)

  for (i in 1:1000){
    boot_binom = sample(x_binom, size = n, replace = TRUE)
    binom_n1[i] = var(boot_binom)
    binom_n[i] = (n-1)/n * var(boot_binom)
  }

  norm_n = rep(NA, 1000)
  norm_n1 = rep(NA, 1000)

  for (i in 1:1000){
    boot_norm = sample(x_norm, size = n, replace = TRUE)
    norm_n1[i] = var(boot_norm)
    norm_n[i] = (n-1)/n * var(boot_norm)
  }

  expo_n = rep(NA, 1000)
  expo_n1 = rep(NA, 1000)

  for (i in 1:1000){
    boot_expo = sample(x_expo, size = n, replace = TRUE)
    expo_n1[i] = var(boot_expo)
    expo_n[i] = (n-1)/n * var(boot_expo)
  }

  results = bind_rows(
    tibble(sample_size = n, distribution = "Poisson", estimator = "S2_n-1", estimate = pois_n1),
    tibble(sample_size = n, distribution = "Poisson", estimator = "S2_n", estimate = pois_n),
    tibble(sample_size = n, distribution = "Binomial", estimator = "S2_n-1", estimate = binom_n1),
    tibble(sample_size = n, distribution = "Binomial", estimator = "S2_n", estimate = binom_n),
    tibble(sample_size = n, distribution = "Normal", estimator = "S2_n-1", estimate = norm_n1),
    tibble(sample_size = n, distribution = "Normal", estimator = "S2_n", estimate = norm_n),
    tibble(sample_size = n, distribution = "Exponential", estimator = "S2_n-1", estimate = expo_n1),
    tibble(sample_size = n, distribution = "Exponential", estimator = "S2_n", estimate = expo_n)
  )

  return(results)
}
```

```r

bootstrap_results = bind_rows(
  boot_by_size(20),
  boot_by_size(50),
  boot_by_size(100),
  boot_by_size(500)
)
```

```r
ggplot(bootstrap_results, aes(x = estimate, fill = estimator)) +
  geom_density(alpha = 0.4) +
  facet_grid(sample_size ~ distribution, scales = "free") +
  geom_vline(xintercept = true_val, linetype = "dashed") +
  xlab("Estimated Variance") +
  ylab("Density") +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
```

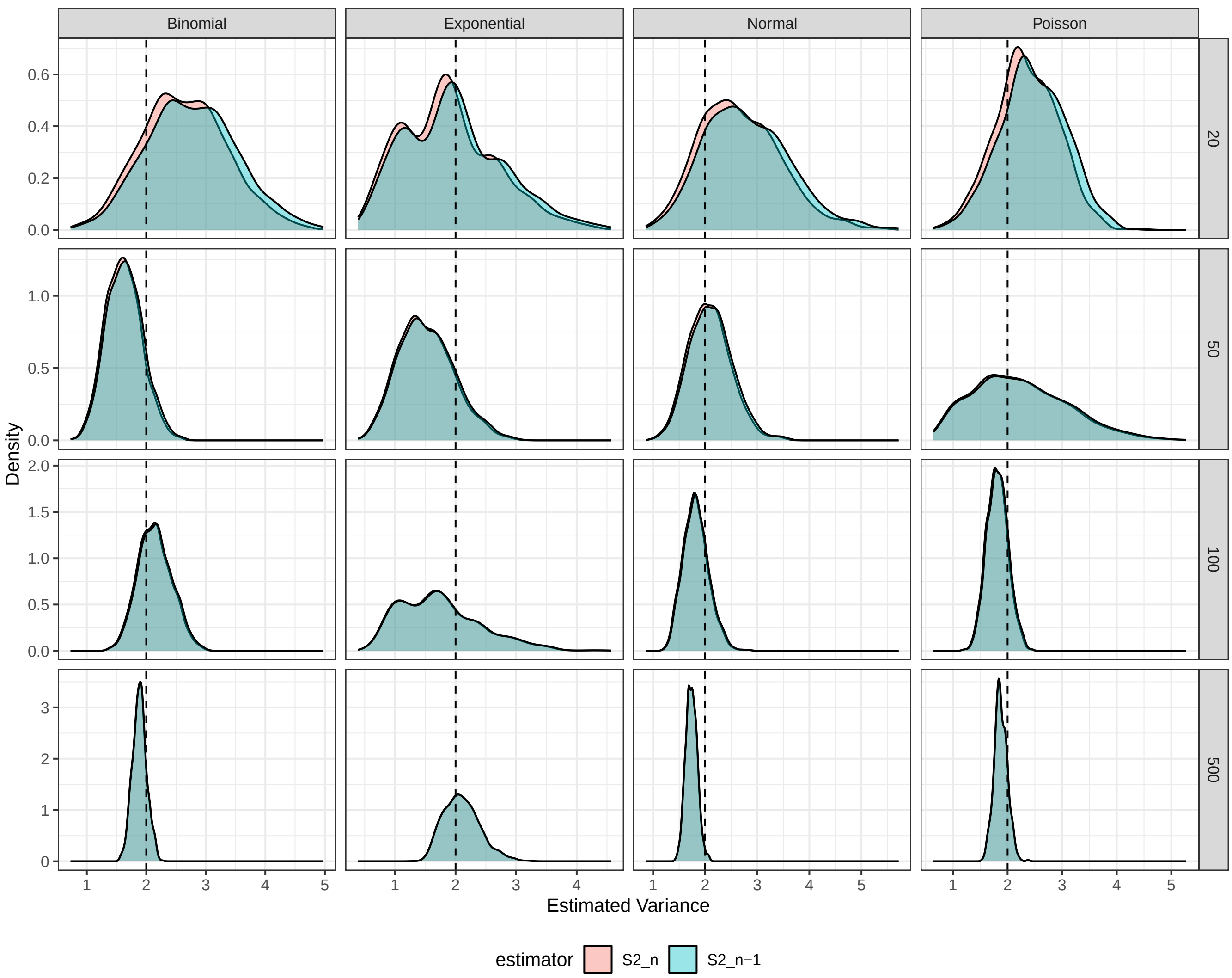
Figure 4: Bootstrap Density Plots of Variance Estimators by Sample Size (Seed: 7272)



```r
set.seed(5055)

bootstrap_results = bind_rows(
  boot_by_size(20),
  boot_by_size(50),
  boot_by_size(100),
  boot_by_size(500)
)
```

```r
ggplot(bootstrap_results, aes(x = estimate, fill = estimator)) +
  geom_density(alpha = 0.4) +
  facet_grid(sample_size ~ distribution, scales = "free") +
  geom_vline(xintercept = true_val, linetype = "dashed") +
  xlab("Estimated Variance") +
  ylab("Density") +
  theme_bw() +
  theme(
    legend.position = "bottom"
  )
```

Figure 5: Bootstrap Density Plots of Variance Estimators by Sample Size (Seed: 5055)



Because we are resampling with replacement, we are going to get a slightly different sample distribution, easily shown in the diffrent graphs of figure 3 and 4. Our bootstrap gives generally the same idea as in our full simulation. In both ideas, in almost all trials, the $S^2_{n-1}$ is centered more around the true value compared to $S^2_n$. In fact, $S^2_n$ seems to consistently be left shifted, showing how it has a decrease bias. Once again, we see that both distributions become more tightly and better centered around the true value as our sample size increases.

However, we do see results vary from seed to seed. This is because our bootstrap resampling does not resample from the true population but instead samples with replacement from the original sample. That original sample is dependent on our seed, thus we will see distinctions between seeds. Overall however, the same qualititative behavior exists between seeds so our analysis is stil valid.

# 3 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) summary of the simulation and the results using your work in Lab 11. Your summary should be professional, clear, and all claims should be corroborated with statistical support.